ChatGPT said:

RAG — Simple Notes for Learning & Quick Revision

These notes cover your RAG learning journey from the basics to your practical implementation using **Python, Gemini, LangChain, and Qdrant**, including important tricky points and production considerations.

The goal is to make these notes easy to revise before interviews and useful when you build another RAG application.

1. What is RAG?

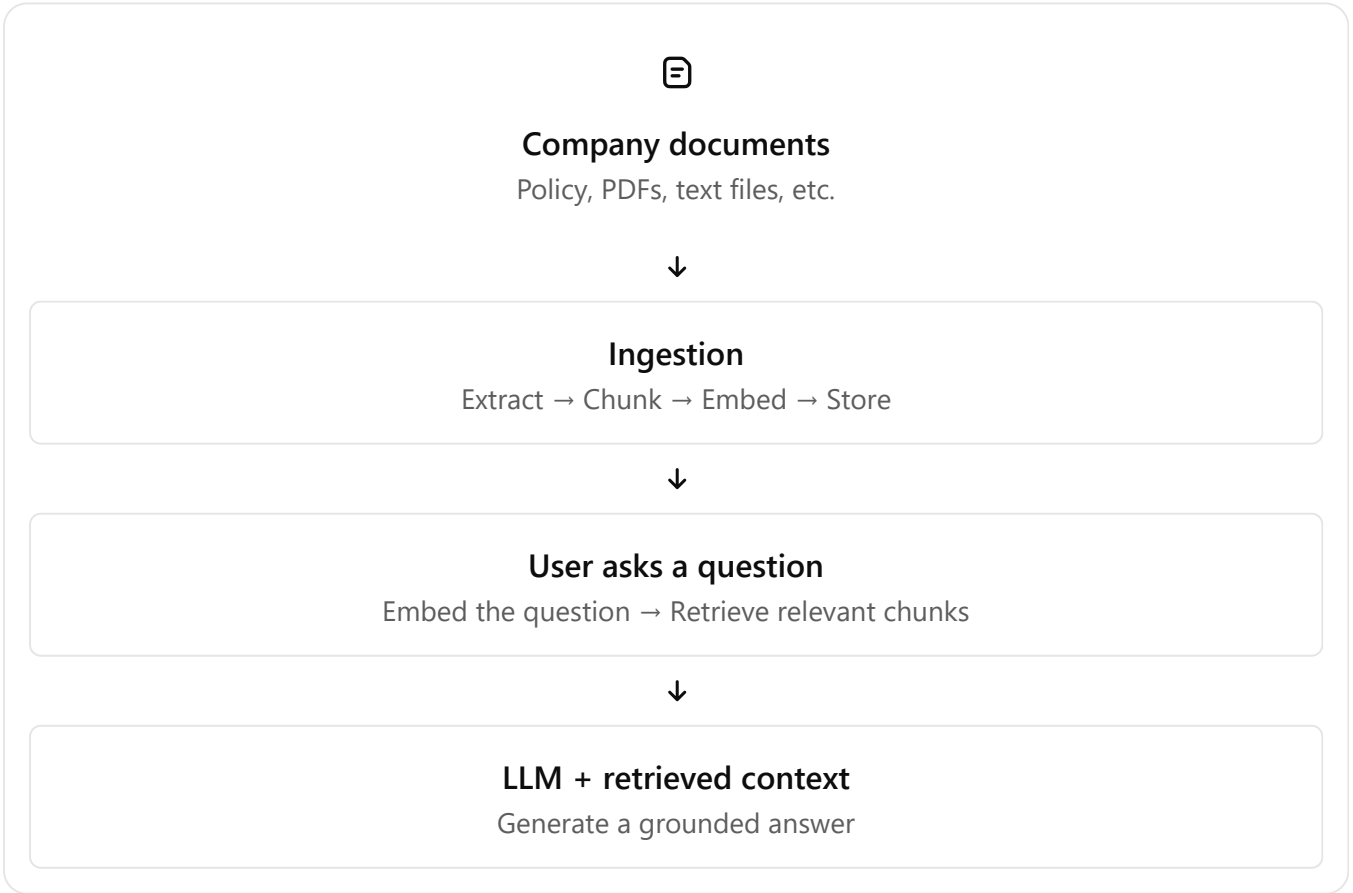
RAG = Retrieval-Augmented Generation

RAG is a technique in which an LLM retrieves relevant information from an external knowledge source and uses that information to generate an answer.

Example: Suppose your company policy says employees get 24 annual leave days.

Without RAG, the LLM might not know your company's policy and could guess.

With RAG, your application retrieves the relevant policy text and gives it to the LLM as context.



Why use RAG?

- Answer questions about private or company-specific documents.
- Provide answers grounded in retrieved information.
- Update knowledge without retraining the LLM.
- Include source information to help users verify answers.

Important: RAG can reduce hallucinations, but it does not guarantee that every answer is correct.

2. Important RAG terminology

Term	Simple meaning
LLM	Model that understands prompts and generates responses, e.g. Gemini
Document	Original information, such as a policy file
Chunk	A smaller piece of a document
Embedding	A numerical vector representing text
Vector database	Stores vectors and supports similarity search, e.g. Qdrant
Similarity search	Finds vectors close to the query vector
Retrieval	Fetches relevant chunks
Context	Information supplied to the LLM to answer the question
Ingestion	Preparing and storing documents for retrieval
Reranking	Reordering retrieved candidates by relevance
Metadata	Additional information such as filename, department or page
Top-K	Maximum number of results requested from retrieval

3. The complete RAG workflow

RAG has two main stages.

Stage A — Ingestion (usually before questions arrive)

1. Load documents.
2. Split documents into chunks.
3. Generate an embedding for each chunk.

4. Store vectors, chunk text and metadata in Qdrant.

Stage B — Retrieval and generation (for each question)

1. Receive the user's question.
2. Generate an embedding for the question.
3. Search Qdrant for similar vectors.
4. Optionally filter and rerank the results.
5. Send the best chunks and the question to the LLM.
6. Return an answer, ideally with sources.

Tricky point: In a normal RAG application, you do not re-ingest and re-embed every document for every question. Ingestion happens separately, and retrieval happens for each query.

4. Chunking — splitting documents

LLMs cannot always use an entire large document efficiently. We split the document into smaller pieces called **chunks**.

Example document:

```
Annual leave: 24 days per year.
Sick leave: 12 days per year.
Employees must request planned leave 3 working days in advance.
```

Possible chunks:

```
Chunk 1: Annual leave: 24 days per year.

Chunk 2: Sick leave: 12 days per year.

Chunk 3: Employees must request planned leave 3 working days in advance.
```

Chunk size and overlap

Your LangChain configuration:

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
```

```
    chunk_overlap=80,
    separators=["\n\n", "\n", ". ", " ", ""],
)
```

- `chunk_size=500` : targets chunks of up to 500 **characters**, not tokens.
- `chunk_overlap=80` : adjacent chunks can share 80 characters.
- `separators` : tells the splitter which boundaries to try first.
- `split_text(text)` : returns a list of text chunks.

Illustration of overlap

Chunk 1: A B C D E F G H

Chunk 2: G H I J K L M N

G and H appear in both chunks to preserve context at the boundary. Real chunks overlap by characters according to the configured size.

Tricky chunking points

- **Very small chunks:** may lose context and split useful facts apart.
- **Very large chunks:** may include irrelevant information and consume more context tokens.
- **Overlap:** helps preserve information across boundaries, but creates duplicated text and additional storage.
- `RecursiveCharacterTextSplitter` tries larger separators first, then smaller ones when necessary. A separator such as `". "` does not guarantee every sentence remains intact.
- Chunk size is not a universal value. Tune it based on document type, retrieval quality and model context limits.
- Tables, code, PDFs and structured documents may need specialized parsing or chunking.

Practical rule: Start with a reasonable chunk size, inspect the actual chunks, and evaluate them using realistic questions. Don't select chunk settings based on a fixed rule alone.

5. Embeddings — converting text into vectors

An embedding model converts text into a list of numbers called a **vector**.

Example (illustrative values only):

```
text = "Employees receive 24 annual leave days."
```

```
vector = get_embedding(text)

print(vector)
# [0.12, -0.31, 0.08, ...]
```

Your application uses Gemini:

```
result = gemini.models.embed_content(
    model="gemini-embedding-001",
    contents=text,
)

vector = result.embeddings[0].values
```

What do embeddings represent?

They encode patterns of meaning learned by the embedding model. Texts with related meanings may have vectors that are close in vector space.

For example:

- "How many annual leave days do employees receive?"
- "What is the yearly vacation allowance?"

The wording differs, but a good embedding model may represent their meanings similarly.

Tricky embedding points

1. **An embedding is not a keyword list.** It is a numerical representation learned by a model.
2. **Similar meaning does not mean identical vectors.** Related questions can still have different vectors.
3. **Dimensions are not human-readable concepts.** You generally cannot say one dimension means "leave" and another means "salary."
4. **Use compatible embedding models and configurations** for documents and queries. If you change models, you will usually need to re-embed the documents.
5. **Vector dimensions must match** the collection's configured dimensions.
6. An embedding is not the final answer. It is used to find relevant text; the LLM generates the answer later.

6. Vector databases and Qdrant

A vector database stores vectors and allows similarity search. Qdrant stores vectors along with optional payload metadata.

Think of a stored Qdrant point as:

```
{
    "id": "unique-point-id",
    "vector": [0.12, -0.31, 0.08],
    "payload": {
        "text": "Employees receive 24 annual leave days.",
        "source": "company_policy.txt",
        "chunk_index": 0,
        "department": "HR"
    }
}
```

The vector above is illustrative; actual embeddings contain many more dimensions.

Qdrant concept	Purpose
Collection	Groups points, similar to a table at a conceptual level
Point	One stored record containing a vector and optional payload
Vector	Numerical representation of a chunk
Payload	Chunk text and metadata
Point ID	Identifies a stored point
Distance metric	Measures how close vectors are
Payload index	Helps metadata-filtered searches

Your collection configuration

```
qdrant.create_collection(
    collection_name="company_policy_chunks",
    vectors_config=models.VectorParams(
        size=len(vector),
        distance=models.Distance.COSINE,
    ),
)
```

- `size` must match the embedding vector length.
- `COSINE` specifies the similarity/distance metric used for search.
- The collection must be created before inserting points.

Tricky point: If the collection already exists, check that its vector configuration is compatible with the embedding model you are using. A matching dimension alone does not guarantee that vectors from different embedding models are comparable.

7. Similarity search and cosine similarity

During retrieval, the question is embedded and compared against the stored chunk vectors.

For example:

```
query_vector = get_embedding(
    "How much annual leave do I get?"
)

results = qdrant.query_points(
    collection_name=COLLECTION_NAME,
    query=query_vector,
    limit=5,
    with_payload=True,
).points
```

Qdrant finds the nearest vectors according to the collection's distance configuration.

Cosine similarity — intuition

Cosine similarity compares the angle between two vectors.

- A value closer to `1` generally means more similar directions.
- A value around `0` indicates little directional similarity.
- A value closer to `-1` indicates opposite directions in the mathematical cosine measure.

For normalized vectors, the cosine similarity is the dot product of the vectors.

Important: Qdrant's returned `point.score` is the score associated with the configured distance metric. For cosine search, a higher score generally means greater similarity. Do not assume every vector database returns scores with the same interpretation.

Top-K

`limit=5` requests up to five results.

- Too few results may miss useful context.
- Too many results may include noise, duplicates and unnecessary tokens.
- Top-K is a maximum, not a promise that exactly that many relevant results exist.

For your improved workflow, you used `TOP_K = 10` for candidates, followed by reranking to keep the best three.

8. Score threshold — filtering weak matches

```
results = qdrant.query_points(
    collection_name=COLLECTION_NAME,
    query=query_vector,
    limit=10,
    score_threshold=0.60,
    with_payload=True,
).points
```

A score threshold rejects results that do not meet the configured minimum.

Example: If the question is about annual leave but the nearest available chunks discuss office parking, a threshold may help exclude weak matches.

Tricky points:

- `0.60` is only an initial experimental value, not a universal RAG standard.
- A strict threshold can remove relevant results.
- A loose threshold can allow irrelevant results.
- Similarity scores do not guarantee factual correctness.
- Test questions with known answers and questions whose answers are absent from your documents.

If no chunks remain, return a message such as:

> I couldn't find enough relevant information in the indexed documents.

That is usually better than generating an unsupported answer.

9. Metadata and payload filtering

Metadata describes a chunk without being its main content.

Example:

```
payload = {
    "text": "Employees receive 24 annual leave days.",
    "source": "HR/leave_policy.txt",
    "department": "HR",
    "chunk_index": 0
}
```

Filter retrieval by department:

```
query_filter = models.Filter(
    must=[
        models.FieldCondition(
            key="department",
            match=models.MatchValue(value="HR"),
        )
    ]
)
```

Then pass it into `query_points`:

```
results = qdrant.query_points(
    collection_name=COLLECTION_NAME,
    query=query_vector,
    query_filter=query_filter,
    limit=5,
    with_payload=True,
).points
```

Why metadata matters

- Search only HR documents.
- Restrict results to a specific source file.
- Track which document and chunk produced an answer.
- Support access-control filtering when implemented securely.

Tricky points

- A payload index helps filtering performance; it does not improve the semantic quality of the embedding.

- A filter restricts the candidate search space. It does not itself rank text by meaning.
- The metadata values must match the values actually stored in Qdrant.
- In a multi-tenant application, derive tenant identity from authenticated server-side context. Never trust a client-provided tenant ID to enforce access control.

10. Ingestion — storing chunks correctly

Your ingestion process is responsible for getting documents into Qdrant.

A simplified example:

```
chunks = load_and_chunk_document(DOCUMENT_PATH)

points = []

for index, chunk in enumerate(chunks):
    vector = get_embedding(chunk)

    points.append(
        models.PointStruct(
            id=str(uuid.uuid4()),
            vector=vector,
            payload={
                "text": chunk,
                "source": DOCUMENT_PATH.name,
                "chunk_index": index,
            },
        )
    )

qdrant.upsert(
    collection_name=COLLECTION_NAME,
    points=points,
    wait=True,
)
```

What does `upsert` mean?

`upsert` inserts a point or updates the existing point if the same ID already exists.

Tricky point: If you generate random UUIDs every time you ingest a file, rerunning ingestion creates new points rather than updating old ones. This causes duplicate chunks.

Your improved code uses deterministic IDs based on document version and chunk information, which helps avoid duplication during repeated ingestion of the same version.

11. Incremental ingestion and document versioning

You added content hashes to avoid re-embedding unchanged documents.

```
content_hash = hashlib.sha256(
    text.encode("utf-8")
).hexdigest()
```

A content hash changes when the text changes.

You can also include chunking and embedding settings:

```
version_text = (
    f"{content_hash}|{CHUNK_SIZE}|"
    f"{CHUNK_OVERLAP}|{EMBEDDING_MODEL}"
)

version_hash = hashlib.sha256(
    version_text.encode("utf-8")
).hexdigest()
```

This means changing the chunk size or embedding model can trigger re-ingestion even if the source document is unchanged.

Scenario	Expected behavior
New file	Ingest and embed it
Unchanged file	Skip embedding work
Changed file	Generate new chunks and vectors
Deleted file	Remove its old chunks
Changed chunking settings	Re-ingest affected documents
Changed embedding model	Re-embed with the new model

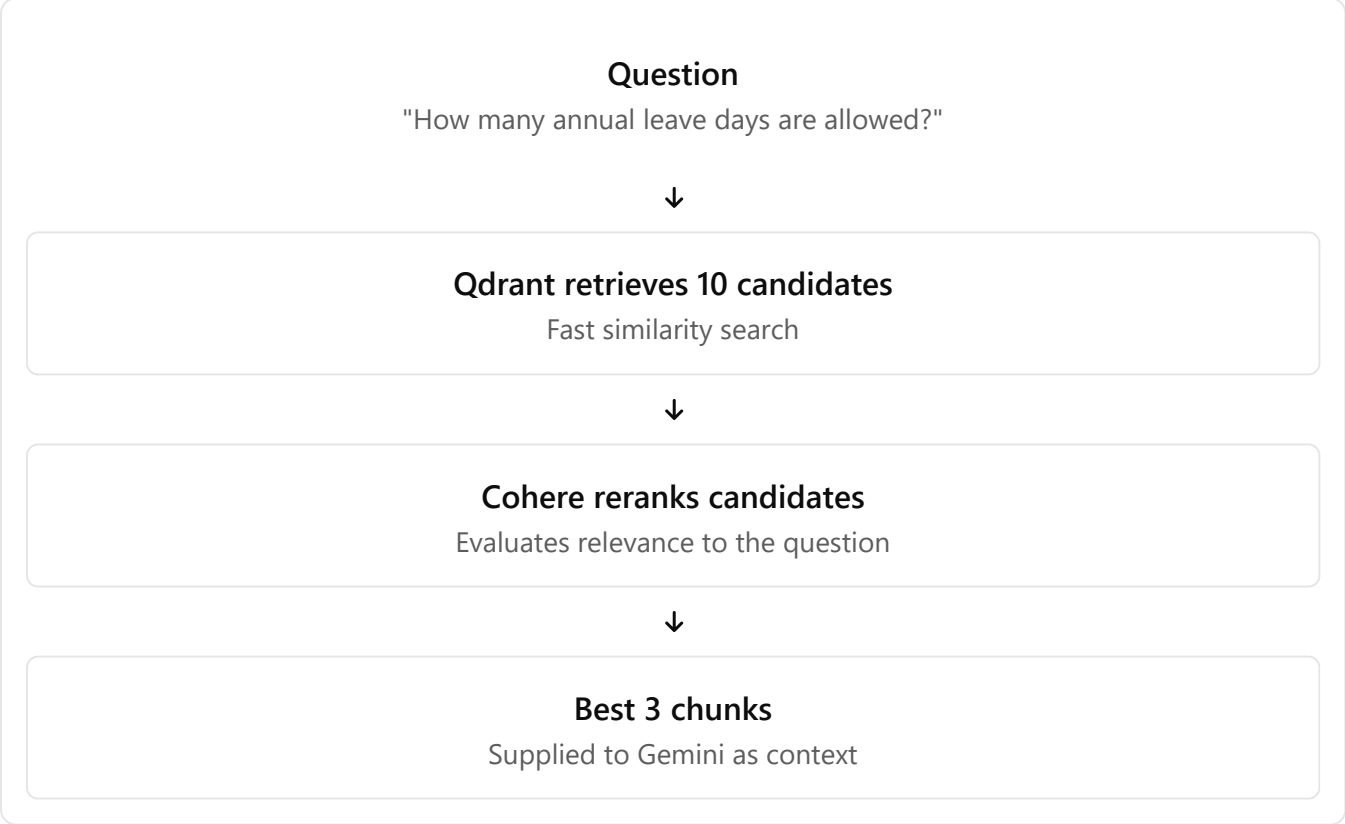
Tricky production points

- Keep ingestion state somewhere durable.

- A local JSON state file is fine for learning, but concurrent workers and multiple application instances need coordinated state management.
- If a file is deleted, remove its points from Qdrant as well; otherwise, obsolete information can still be retrieved.
- If you use the same collection for multiple document sets, make sure cleanup is scoped correctly.
- If ingestion fails halfway through, your system must not mark the document as successfully processed prematurely.

12. Reranking — improving retrieval quality

The initial vector search is designed to retrieve promising candidates quickly. A reranker then evaluates the question against those candidates and reorders them.



Example code:

```
response = cohere_client.rerank(
    model="rerank-v4.0-pro",
    query=question,
    documents=candidate_texts,
    top_n=3,
)
```

Why not rerank everything?

Vector search narrows a large collection to a manageable set. Reranking that smaller set can improve relevance without reranking every stored chunk.

Tricky points

- Reranking cannot recover a relevant chunk that initial retrieval never found.
- Reranking improves ordering; it does not guarantee correctness.
- Rerankers introduce additional latency, cost and an external dependency.
- When Cohere is not configured, your fallback uses Qdrant's ranking. **That is not reranking.**
- Reranking scores are not directly interchangeable with Qdrant similarity scores.

13. Generation — asking the LLM to answer

After retrieval, you combine the question with the retrieved context.

```
prompt = f"""
Answer the question using only the context below.
If the answer is missing, say you do not have enough information.
Do not invent facts.

Context:
{context}

Question:
{question}
"""

response = gemini.models.generate_content(
    model=GENERATION_MODEL,
    contents=prompt,
)
```

Why pass the context?

The LLM can use your private or current document content rather than relying only on its pre-trained knowledge.

Prompt-writing best practices

- Clearly separate the context and the question.
- Tell the model what to do when the answer is missing.

- Ask it to cite the source or document name.
- Avoid placing unnecessary chunks into the prompt.
- Treat retrieved documents as **untrusted data**, not instructions. A malicious document could contain prompt-injection text.

Tricky point: Prompt instructions help but are not a complete security boundary. Validate permissions before retrieval, and do not allow retrieved text to independently authorize sensitive actions or tool calls.

14. What causes poor RAG answers?

Not every RAG problem is a generation problem.

Symptom	Possible cause
Correct answer is missing	Relevant chunk was not retrieved
Wrong document appears	Similarity search or metadata filter is unsuitable
Answer lacks important details	Chunk is too small or incomplete
Answer contains irrelevant details	Chunk is too large or retrieval returns noisy results
Correct chunk ranks too low	Consider reranking or improving retrieval
Answer invents facts	Context is insufficient or generation instructions are weak
Old policy appears	Ingestion or deletion cleanup failed
Correct text but poor answer	Prompt, context organization or generation issue

A simple debugging strategy

For every test question, print:

1. The original question.
2. The retrieved chunk text.
3. The source filename and chunk index.
4. The Qdrant similarity scores.
5. Reranking scores, if reranking is enabled.
6. The final prompt context and generated answer.

This helps you identify whether the issue is in ingestion, retrieval, ranking or generation.

15. RAG evaluation — how do you know it works?

Don't evaluate your RAG application only by asking a few questions and deciding the answers look good.

Create a small test dataset:

Question	Expected information	What to check
How many annual leave days?	24 days	Was the correct chunk retrieved?
How much sick leave?	12 days	Did the answer match the document?
Can unused leave be carried forward?	Maximum 10 days	Was the relevant policy retrieved?
What is the office parking fee?	Not specified	Did the model avoid inventing an answer?

Evaluate three areas separately:

- **Retrieval quality:** Did the system retrieve the relevant information?
- **Answer quality:** Is the answer correct and supported by the context?
- **Operational quality:** Are latency, token usage, cost and failures acceptable?

A useful retrieval metric is *Recall@K*: whether a relevant chunk appears within the first K retrieved results. For example, Recall@5 asks whether the expected relevant chunk appears among the first five results.

16. Production best practices

Your learning application is a good foundation. Before using it for real users, consider these points.

- **Embedding consistency:** use compatible embedding models and settings for ingestion and queries.
- **Incremental ingestion:** detect changed files, avoid duplicate points and remove stale chunks.
- **Metadata indexes:** add indexes for fields used in filters.
- **Access control:** enforce user and tenant permissions on the server before returning retrieved context.
- **Prompt injection:** treat retrieved content as untrusted input.
- **Evaluation:** test retrieval and answer quality with known questions.
- **Observability:** log latency, retrieval scores, sources and failures without exposing sensitive information.

- **Cost control:** monitor embedding calls, reranking calls, generation tokens and unnecessary re-ingestion.
- **Reliability:** plan for API timeouts, rate limits, partial ingestion and service outages.
- **Scaling:** use durable ingestion state and coordinated workers when multiple instances process documents.

For a small learning project, a local text file plus Qdrant is reasonable. For a production service, you may need a database-backed ingestion registry, a background job queue, authentication, monitoring and automated tests.

17. Common interview questions — quick answers

Q1. What is the difference between RAG and fine-tuning?

RAG retrieves external information at query time. Fine-tuning updates model behavior by training on examples. RAG is commonly useful for changing document knowledge; fine-tuning is useful when you need to adapt behavior, style or task performance.

Q2. Why do we split documents into chunks?

To retrieve focused pieces of information instead of passing entire documents into the LLM. Chunk size and overlap affect context, retrieval quality and cost.

Q3. Why do we embed the question?

To represent the question in vector space so the vector database can search for semantically similar document chunks.

Q4. Why use a vector database such as Qdrant?

It stores embeddings and supports nearest-neighbor search, along with payload metadata and filtered retrieval.

Q5. What is the difference between top-K retrieval and reranking?

Top-K selects a limited number of candidates from the initial search. Reranking reorders those candidates according to their relevance to the question.

Q6. Does RAG eliminate hallucinations?

No. It provides supporting context, but retrieval can fail and the LLM can still produce unsupported answers.

Q7. Why not store the entire document as one embedding?

One vector represents the whole document rather than individual facts. A query may match the document generally but fail to identify the exact passage needed to answer the question.

Q8. What happens when a document changes?

Detect the changed content, split and embed the new version, update the stored points and remove obsolete chunks.

Q9. Why use metadata filtering?

To narrow the search to an appropriate source, department, tenant or document category. Metadata filtering complements semantic search; it does not replace it.

Q10. Is LangChain mandatory for RAG?

No. LangChain provides reusable components for splitting, retrieval and orchestration. You can build the same workflow directly using Gemini and Qdrant SDKs. Choose based on maintainability, integration needs and complexity.

18. One-minute revision sheet

Flashcards

0 of 15 revealed

RAG

Ingestion

Query flow

Chunk size

Overlap

Embedding

Qdrant

Cosine

Top-K

Threshold

Metadata

Reranking

Hashing

Grounded answer

Evaluation

Reveal all answers

My recommendation: save these notes and revise the flashcards first. Then practise explaining your own Gemini + Qdrant implementation from memory, especially **why chunking, embeddings, top-K, thresholding, metadata filtering, reranking and incremental ingestion are needed**. Being able to explain those design choices is a strong step toward building production-quality RAG systems.

If you want, I can:

- Explain the importance of ingestion state management
- Detail the reranking process and tradeoffs